



**Instituto de Tecnología e Ingeniería
Lic. en Informática**

**Informe técnico
Diseño y desarrollo de
Sistema backend para red social
“UnaHur Anti-Social Net”**

Trabajo Práctico N° 1

**-Materia: Estrategias de persistencia.
-Profesor: Lucas Figarola.
-Ciclo lectivo: 2026 - 1° cuatrimestre.**

**-Integrantes:
Albarracin, Ary DNI (41.450.573)
Alderete Cuello, Luis Sebastian DNI (37.646.209)
Lopez, Felipe DNI ()
Maldonado, Martín Marcelo DNI (31.554.217)**

Informe técnico (ciclo de vida del software, patrones de diseño y gestión de configuración)

Ingeniería de Software: Ciclo de Vida del Proyecto

Estructuramos cuatro pilares fundamentales:

Gestión de Configuración (SCM),

Arquitectura y Patrones,

Diseño: Modelado de Datos Relacional, y

Validación e Integración (Testing/Docs).

Detalle de Comandos Git Bash: Justificación en Ingeniería de Software

A continuación, se desglosan los bloques de comandos que ejecutamos desde la terminal ([Git Bash](#)) como **administrador**, explicando **para qué** sirve cada uno bajo el prisma de la **Ingeniería de Software** y la **gestión de dependencias**:

Bloque 1: Inicialización del Espacio de Trabajo y Gestión de Dependencias Core

Bash

```
cd "C:\Users\anti-social-relational-tp-unahur-data-systems"  
npm install express sequelize sqlite3 multer dotenv
```

Para arrancar, nos situamos en la raíz del workspace con la terminal para que Git pueda gestionar todo correctamente. El paso clave acá fue ejecutar `npm install`, lo que nos permitió bajar todas las dependencias necesarias y dejarlas registradas en el `package.json`. Elegimos **Express** para manejar el

ruteo, **Sequelize** como ORM para no pelearnos con SQL puro, y **SQLite3** porque al ser embebido nos asegura que el proyecto funcione en cualquier máquina sin configurar nada extra. También sumamos **Multer** para la subida de archivos y **Dotenv** para no dejar las credenciales y secretos pegados en el código, siguiendo lo que dictan las buenas prácticas.

- El cambio de directorio ubica la terminal a raíz del *workspace* gestionado por Git.
- `npm install` descarga e incorpora las dependencias del ecosistema al manifiesto del proyecto (`package.json`).
- **Justificación de Ingeniería:** Establece las librerías base de producción.
- Se selecciona **Express** como el middleware/framework para el ruteo HTTP, **Sequelize** como la capa de abstracción de datos (Patrón ORM), **SQLite3** como motor ligero de persistencia embebida que garantiza **portabilidad** absoluta, **Multer** para el manejo de streams de datos multipart/form-data (subida de archivos al sistema de archivos del servidor), y **Dotenv** para desacoplar el código de los secretos del entorno operativo, cumpliendo con las buenas prácticas de factoría de software de separar configuración de código duro.

Bloque 2: Entorno de Desarrollo y Herramientas Auxiliares

Bash

```
npm install -D nodemon sequelize-cli
```

- **¿Para qué se hizo?** Instala herramientas exclusivas para la etapa de construcción, guardándose bajo el nodo `devDependencies`.
- **Justificación de Ingeniería:** Automatización del flujo de trabajo. **Nodemon** actúa como un monitor de procesos en caliente que reinicia el servidor ante cambios en el código base, acelerando el ciclo de *feedback* en el desarrollo. **Sequelize-CLI** introduce la automatización por línea de comandos para el andamiaje (*scaffolding*) de la persistencia y la trazabilidad del esquema de datos.

Bloque 3: Generación de Entidades.

```
npx sequelize-cli init
```

```
npx sequelize-cli model:generate --name Usuario --attributes nickName:string
```

```
npx sequelize-cli model:generate --name Post --attributes descripcion:text,fechaCreacion:date
```

```
npx sequelize-cli model:generate --name PostImage --attributes urlImagen:string
```

```
npx sequelize-cli model:generate --name Comentario --attributes texto:text,fecha:date
```

```
npx sequelize-cli model:generate --name Etiqueta --attributes nombre:string
```

- **¿Para qué se hizo?:** Genera la estructura de carpetas estándar (**models**, **migrations**, **seeders**, **config**) e inicializa los esqueletos físicos para las tablas de la base de datos.
- **Justificación de Ingeniería:** Estándar de diseño. En lugar de estructurar de forma caótica, el CLI fuerza convenciones de nomenclatura. Cada comando **model:generate** crea en paralelo el archivo del **Modelo JavaScript** (la representación lógica de la entidad para interactuar mediante código) y el archivo de la **Migración** (el código SQL imperativo que altera la base de datos física).

Bloque 4: Versionado Inicial y Configuración de Trazabilidad

```
git commit -m"instalación de Express y Sequelize. Se agregó el driver de SQLite. Se instaló dependencia de desarrollo nodemon sequelize-cli inicializada. Se crearon las Entidades"
```

- **¿Para qué se hizo?:** Crea el primer punto de control (*checkpoint*) en el historial local de Git.
- **Justificación de Ingeniería:** Gestión de la Configuración de Software (SCM). Establece una línea base limpia de la infraestructura del proyecto antes de alterar la estructura lógica o física del almacenamiento, asegurando la reversibilidad del código.

Bloque 5: Ejecución del Esquema Físico (Migración e Integridad Relacional)

```
npx sequelize-cli db:migrate
```

- **¿Para qué se hizo?:** Toma los archivos generados en la carpeta `migrations` y los ejecuta de manera secuencial directamente sobre el binario local `database.sqlite`.
- **Justificación de Ingeniería:** Evolución controlada del esquema de datos. En Ingeniería de Software, las migraciones son el "control de versiones" pero para las tablas de la base de datos. Garantiza que cualquier desarrollador del equipo (como Katia o los colegas del centro de estudiantes) pueda clonar el repositorio, correr este comando y poseer exactamente la misma estructura de tablas con sus restricciones relacionales e integridad referencial idénticas en su entorno local.

Bloque 6: Versionado de la Capa de Persistencia y Despliegue en Remoto

```
git commit -m "Capa de persistencia sqlite con sequelize..."
```

```
git push
```

```
git add .
```

- **¿Para qué se hizo?:** Confirma la estructura relacional estable e impacta la rama local hacia el servidor remoto (GitHub).
- **Justificación de Ingeniería:** Integración Continua (CI) y Respaldo. Al subir el código a la organización `EP-UnaHur-2026C1` en GitHub, el incremento de software queda disponible para revisión por pares, auditoría de código o disparadores automatizados de testing.

Bloque 7: Integración de la Interfaz de Documentación Dinámica

```
npm install swagger-ui-express swagger-jsdoc
```

```
npm run dev
```

- **¿Para qué se hizo?:** Instala e integra las librerías necesarias para transformar los bloques de comentarios JSdoc del código en especificaciones OpenAPI/Swagger, levantando el entorno local mediante el script reactivo del `package.json` (`npm run dev`).
- **Justificación de Ingeniería:** Documentación de API basada en el principio *Code-First*. Al exponer el contrato de la interfaz en `http://localhost:3000/api-docs/`, se elimina la necesidad de documentar manualmente en archivos externos desactualizados. El backend se vuelve auto-documentado, interactivo y transparente para los equipos que consuman la API en el futuro (Frontend, QA, etc.).

3. Resumen Técnico del recorrido

Resumen secuencial del ciclo de desarrollo de la solución:

1. **Definición de Requisitos y Arquitectura Base:** Se partió del enunciado del TP para una red social orientada a datos relacionales. Optamos por una **arquitectura en capas** (Rutas -> Controladores -> Modelos/ORM -> Base de Datos) montada sobre Node.js, aislando las responsabilidades del sistema.
2. **Modelado Entidad-Relación y Restricciones:** Generamos las entidades clave mediante el CLI de Sequelize. Modificamos los archivos resultantes para asegurar las **reglas de integridad**: obligatoriedad de la descripción en los posts, claves únicas en los nicknames de usuarios (`unique: true`), y establecimos relaciones Uno a Muchos (Post a PostImage, Usuario a Post, Post a Comentario).
3. **Modelado Avanzado (Relaciones Muchos a Muchos):** Codificamos de manera manual los esquemas relacionales complejos que el CLI no autogenera por defecto: la asociación cruzada N:M entre **Posts y Etiquetas** (mediante tabla pivote) y el requerimiento adicional de **Seguidores / Followers** (relación autorreferencial de la tabla Usuarios consigo misma usando alias cruzados en Sequelize).

4. **Ciclo de Debugging y Control de Calidad (QA):** Al realizar las pruebas funcionales dinámicas utilizando la interfaz visual de Swagger, detectamos e interceptamos desviaciones del comportamiento esperado:
 - *Corrección en Comentarios:* Solventamos una excepción de violación de nulos (`notNullViolation`) remapeando el campo `contenido` del request HTTP hacia la columna física `texto` definida en el ORM.
 - *Corrección en Feed:* Reparamos un fallo de ejecución SQL en el endpoint `GET /api/posts` cambiando un criterio de ordenamiento erróneo basado en una columna ficticia (`fechaCreacion`), normalizándolo bajo la estampa temporal nativa `createdAt`.
 - *Garantía de Left Join:* Configuramos la propiedad `required: false` en la inclusión de comentarios del listado, asegurando que la API liste todas las publicaciones existentes en la base de datos (con o sin comentarios cargados).
5. **Configuración Dinámica y Optimización de Rendimiento:** Se acopló el backend a variables de entorno para procesar filtros en tiempo de ejecución (ocultando comentarios que excedan la antigüedad parametrizada en el `.env`). Adicionalmente, formulamos la justificación teórica para implementar estrategias de optimización de cara a una defensa, describiendo mecanismos de caching intermedio (**Redis**) e índices en columnas de alta consulta para mitigar los accesos directos al disco rígido de la base de datos.
6. **Cierre de Versión:** Utilizando Git Bash, empaquetamos el incremento de software funcional libre de bugs, aplicando políticas de confirmación semántica claras y dejando el repositorio listo para producción.

La instalación, la configuración en código y la ejecución del test.

1. Instalación de la Infraestructura

Para incorporar Swagger al ecosistema de Express, corrimos en Git Bash:

```
npm install swagger-ui-express swagger-jsdoc
```

swagger-jsdoc: Es la librería encargada de leer bloques de comentarios estructurados (en formato JSDoc) directamente en nuestro código de rutas y convertirlos en un objeto JSON compatible con el estándar OpenAPI.

swagger-ui-express: Es el middleware que toma ese objeto JSON y genera automáticamente la interfaz web interactiva con los botones y formularios listos para usar.

2. Implementación en Código

El montaje se estructuró en dos partes para mantener limpio el proyecto:

A. Archivo de Configuración (**src/docs/swaggerSpec.js**)

Creamos la especificación base definiendo los metadatos de la API y los archivos donde el framework tiene que ir a buscar los endpoints

JavaScript

```
const swaggerJSDoc = require('swagger-jsdoc');

const options = {
  definition: {
    openapi: '3.0.0',
    info: {
      title: 'Anti-Social-Net API UNaHur',
      version: '1.0.0',
      description: 'Documentación de la capa de persistencia relacional'
    },
    servers: [{ url: 'http://localhost:3000' }],
  },
  apis: ['./src/routes/*.js'] // <--- Le indica dónde leer los comentarios JSDoc
};

module.exports = swaggerJSDoc(options);
```

B. Montaje en el Servidor Principal (**src/main.js**)

Vinculamos las librerías al flujo de Express para exponer la URL pública:

JavaScript

```
const swaggerUi = require('swagger-ui-express');
const swaggerSpec = require('./docs/swaggerSpec');

// Exponer la interfaz visual en la ruta /api-docs
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));
```

A partir de acá, en cada archivo de rutas (como `postRoutes.js`), usamos comentarios especiales para documentar qué hace el método, qué parámetros recibe y qué códigos HTTP devuelve (ej: `200 OK`, `201 Created`, `404 Not Found`).

3. Ejecución de un Test Funcional

Una vez que levantamos el servidor local (`npm run dev`), el ciclo de testeo se realiza directamente desde el navegador web:

Levantamos el servidor local

```
npm run dev
```

1. **Ingreso a la Interfaz:** Entramos a `http://localhost:3000/api-docs` para visualizar el catálogo interactivo dividido por etiquetas (Usuarios, Red Social, Publicaciones, Comentarios).
2. **Selección del Endpoint:** Desplegamos la operación que queríamos validar (por ejemplo, el método `POST /api/comentarios`).
3. **Activación del Formulario:** Hacemos clic en el botón **"Try it out"** (Probar). Esto habilita la edición del cuerpo de la petición (`Request body`) en tiempo real.

4. **Inyección de Datos y Ejecución:** Rellenamos el JSON de prueba con los IDs relacionales del sistema y el mensaje:

```
{  
  "contenido": "¡Excelente diseño de base de datos, compañero!",  
  "usuarioid": 3,  
  "postId": 1  
}
```

Análisis de Respuesta: Al presionar **"Execute"**, Swagger simula un cliente HTTP (como Postman). Manda la petición por red al puerto 3000 y nos devuelve de inmediato la respuesta exacta del servidor de Express (los encabezados, el cuerpo JSON de retorno y el código de estado, permitiéndonos detectar errores como el `notNullViolation` o confirmar éxitos con el `200 OK`).

Cómo desacoplar el almacenamiento físico del almacenamiento lógico

Para procesar **N cantidad** de imágenes sin degradar el rendimiento de la base de datos.

El proceso se estructuró en **tres capas** consecutivas dentro del ciclo de vida de la petición HTTP:

1. Captura y Almacenamiento Físico (Middleware Multer)

Cuando el cliente (Swagger o una app Frontend) envía una publicación con imágenes, no manda un JSON común; envía una petición de tipo `multipart/form-data`.

- **El Proceso:** Configuramos **Multer** como un middleware interceptor en la ruta del `POST /api/posts`. Al usar la directiva `.array('imagenes', N)`, Multer se encarga de recibir el flujo binario de archivos (*stream*), renombrarlos de forma unívoca (para evitar colisiones de nombres si dos usuarios suben un archivo llamado `foto.png`) y guardarlos físicamente en el disco duro del servidor web, dentro de la carpeta local `/uploads`.

- **El Resultado en Código:** Una vez que Multer procesa los archivos con éxito, le inyecta a Express **un array** en el objeto de la petición llamado `req.files`. Si el usuario subió 4 imágenes, `req.files` contendrá 4 objetos con los metadatos de cada archivo (nombre, tamaño, ruta temporal).

2. Mapeo y Persistencia por Lotes (Controlador y Sequelize)

Una vez que las imágenes están a salvo en el disco duro, el flujo pasa a nuestro método `create` en el controlador `postController.js`.

- **El Proceso:** En lugar de hacer inserciones individuales en la base de datos (lo cual generaría múltiples consultas de escritura lentas bloqueando el hilo de ejecución), utilizamos una estrategia de **persistencia por lotes**.
- **El Código:** Recorremos el array de archivos generado por Multer usando un `.map()` para transformar la información binaria en objetos JSON relacionales que contienen únicamente el string con la URL relativa de acceso a la imagen y el ID del post recién creado:

```
if (req.files && req.files.length > 0) {
  const registrosImagenes = req.files.map(file => ({
    urlImagen: `/uploads/${file.filename}`, // <-- Guardamos solo la ruta relativa (el string)
    postId: nuevoPost.id                    // <-- Vinculamos la clave foránea al posteo base
  }));

  // Inserción masiva en una única consulta SQL a SQLite
  await PostImage.bulkCreate(registrosImagenes);
}
```

- **Justificación de Ingeniería:** En la tabla `PostImages` de SQLite **nunca guardamos el archivo binario pesado** (lo que se conoce en bases de datos como un campo *BLOB*), ya que eso influiría el tamaño del archivo de la base de datos y ralentizará los índices. Guardamos únicamente un puntero de texto (`string`) que hace referencia a dónde está el archivo real en el sistema operativo.

3. Recuperación Unificada

Finalmente, para validar que todo quedó perfectamente guardado en la base de datos, cuando se ejecuta el `GET /api/posts`, Sequelize realiza una consulta unificada utilizando **Eager Loading** (Carga asociativa previa).

El ORM hace un `LEFT OUTER JOIN` entre la tabla `Posts` y `PostImages` unificando las entidades a través de la clave foránea `postId`. De esta manera, el servidor responde al cliente con el árbol relacional estructurado, listando las `$$$` imágenes asociadas dentro de un prolijo array de objetos:

```
"imagenes": [  
  
  { "id": 1, "urlImagen": "/uploads/imagenes-1779061483667..." },  
  
  { "id": 2, "urlImagen": "/uploads/imagenes-1779061483723..." },  
  
  { "id": 3, "urlImagen": "/uploads/imagenes-1779061483738..." },  
  
  { "id": 4, "urlImagen": "/uploads/imagenes-1779061483755..." }  
  
]
```

Patrón eficiente de subida de archivos que cuida la salud y la velocidad de procesamiento de la base de datos relacional.

Estrategia de Caché (Bonus 3)

El sistema implementa caché en memoria usando `node-cache` para optimizar la consulta de posts, evitando consultas repetidas a la base de datos.

¿Cómo funciona?

- Primer request**: Se consulta la BD y se guarda el resultado en caché (TTL: 10 min por defecto).
- Requests siguientes**: Se sirven los posts desde memoria hasta que expire el TTL.
- Al crear/eliminar**: Se invalida automáticamente el caché para mostrar cambios inmediatos.

Configuración

En `.env`, se configura el TTL (tiempo de vida en segundos):

```
env
```

```
CACHE_TTL=300
```

Viene predeterminado en 600 segundos(10 minutos) pero se establece en 300 segundos(5 minutos).

¿Cómo ejecutar el proyecto?

1. Instalar dependencias desde la raíz del repositorio (git bash):

```
npm install
```

2. Arrancar el servidor en modo producción:

```
npm start
```

3. Arrancar el servidor en modo desarrollo con reinicio automático:

```
npm run dev
```

4. Una vez iniciado, la documentación Swagger queda disponible en:

<http://localhost:3000/api-docs>

* Si necesitás cambiar el puerto, configura la variable de entorno `PORT` antes de ejecutar la app.*

